

Parity Analysis & Shape Analysis of Unshared Acyclic Lists

Program Analysis and Verification — Final Project

Yarin Klempfner (ID 212171706)

Itamar Klartag (ID 329601538)

September 2026

Outline

① Parity Analysis

- Concrete and Abstract Domains
- Galois Connection and Helper Functions
- Abstract Transfer Functions (ATF)
- Running Example (Formal Step-by-Step)

② Shape Analysis

- Concrete Domain and Shape Graphs
- Normalisation and Abstract Math
- Abstract Transfer Functions (ATF)

③ Implementation & Demo Output

Parity Analysis: Concrete and Abstract Domains

- The concrete domain is $C = (2^\Sigma, \subseteq, \cap, \cup)$ over valuations $\sigma \in \Sigma = \text{Var} \rightarrow \mathbb{N}$.
- The base abstract set is $B = \{Z, E, O\}$, representing $\{0\}$, $\{2, 4, 6, \dots\}$, and $\{1, 3, 5, \dots\}$.
- The abstract domain is $A = \mathcal{P}(\text{Var} \rightarrow B)$.
- It is a disjunctive domain forming a complete lattice of height $3^{|\text{Var}|}$.
- The Galois connection uses $\beta(n)$ to map concrete values to B , defined as $\alpha(S) = \{\beta \circ \sigma \mid \sigma \in S\}$.
- The concretization function is $\gamma(T) = \{\sigma \mid \beta \circ \sigma \in T\}$.

Parity Analysis: Helper Functions

- The abstraction function β is defined as $\beta(n) = E$ if $n \in \text{Even} \setminus \{0\}$, O if $n \in \text{Odd}$, and Z if $n = 0$.
- The successor function $\text{succ}(x)$ models addition by 1: it yields E if $x = O$, and O if $x \in \{E, Z\}$.
- The predecessor function $\text{pred}(x)$ models subtraction by 1: it yields O if $x = E$, Z if $x = Z$, and $\{Z, E\}$ if $x = O$.

Parity Analysis: Abstract Transfer Functions

For every subset t of $T \in A$, the ATFs are grouped by operation type:

Assignments & Arithmetic

- $\llbracket x := y \rrbracket^\# t = \{t[x \mapsto t(y)]\}$
- $\llbracket x := K \rrbracket^\# t = \{t[x \mapsto \beta(K)]\}$
- $\llbracket x := ? \rrbracket^\# t = \{t[x \mapsto b] \mid b \in B\}$
- $\llbracket x := y + 1 \rrbracket^\# t = \{t[x \mapsto b] \mid b \in \text{succ}(t(y))\}$
- $\llbracket x := y - 1 \rrbracket^\# t = \{t[x \mapsto b] \mid b \in \text{pred}(t(y))\}$

Control Flow & Assertions

- $\llbracket \text{assume } E \rrbracket^\# t = \{t\}$ if $t \models^? E$, else $\emptyset$
- $\llbracket \text{assert } ORC \rrbracket^\# t = \{t\}$ if $t \models ORC$, else **err**

Parity Analysis: Running Example Code & Setup

Target Program

L0		n := ?		L1
L1		i := 0		L2
L2		j := 0		L3
L3		assume(i=n)		L6
L3		assume(i!=n)		L4
L4		i := j+1		L5
L5		j := i		L3
L6		assert ...		L7
L7		assert ...		L8

Initialization (Vectors format: (n, i, j))

- Entry at L0 is $\top_A$, comprising all 27 combinations.
- L1 ($n := ?$) keeps all 27 vectors.
- L2 ($i := 0$) sets $i \mapsto Z$, yielding 9 vectors.
- L3 ($j := 0$) sets $j \mapsto Z$. We enter the loop head with:
 $S_0 = \{(Z, Z, Z), (E, Z, Z), (O, Z, Z)\}.$

Parity Analysis: Running Example Iterations

Tracing the loop path $L_3 \rightarrow L_4 \rightarrow L_5 \rightarrow L_3$:

Round 1

- $\text{assume}(i \neq n)$ keeps (E, Z, Z) and (O, Z, Z) .
- $i := j+1$ applies $\text{succ}(Z) = O \implies (E, O, Z), (O, O, Z)$.
- $j := i$ updates $j \implies (E, O, O), (O, O, O)$ (New at L3).

Round 2

- $\text{assume}(i \neq n)$ on new states keeps (E, O, O) .
- $i := j+1$ applies $\text{succ}(O) = E \implies (E, E, O), (O, E, O)$.
- $j := i$ updates $j \implies (E, E, E), (O, E, E)$ (New at L3).

Parity Analysis: Running Example Fixpoint

Round 3 (Fixpoint)

- $\text{assume}(i \neq n)$ on new states keeps (E, E, E) and (O, E, E) .
- $i := j+1$ applies $\text{succ}(E) = O \implies (E, O, E), (O, O, E)$.
- $j := i$ updates $j \implies (E, O, O), (O, O, O)$.
- These vectors are already in state L3. **Fixpoint reached!** (Total: 7 vectors).

Verification Branch (L3 $\rightarrow$ L6)

- $\text{assume}(i=n)$ isolates vectors where i 's class equals n 's class:
 $\{(Z, Z, Z), (E, E, E), (O, O, O)\}$.
- All remaining vectors satisfy the ODD/EVEN assertions at L6 and L7. Program verified.

Shape Analysis: Concrete Domain and Shape Graphs

- Concrete heaps $h = (L, env, n)$ are well-formed if they are unshared and acyclic.
- Abstract length lattice $\mathcal{L} = \{1, O, E\}$ represents exactly 1 step, odd steps ≥ 3 , or even steps ≥ 2 .
- Abstract domain $A = \mathcal{P}(\mathcal{S})$ where $\mathcal{S}$ contains shape graphs $G = (N, env, succ, len, gp)$.
- N represents named nodes; env maps variables to N .
- $len : N \rightarrow \mathcal{L}$ tracks lengths, and $gp : N \rightarrow \{tt, ff\}$ tracks unreachable garbage pointers.
- Galois connection $\alpha(H) = \{\beta(h) \mid h \in H\}$ converts valid concrete heaps to unique shape graphs.

Shape Analysis: Normalisation & Math

- The normalisation function $norm(G)$ removes nodes with no variables.
- **Merge:** If the node has a predecessor p , bypass the node and concatenate lengths:
 $len(p) := len(p) \oplus len(u)$.
- **Garbage Collection:** If the node has no predecessor, delete it, and if it pointed to v , set $gp(v) := tt$ to flag the leak.
- Abstract addition: $1 \oplus 1 = E$, $1 \oplus E = O$, $E \oplus O = O$, etc.
- Abstract subtraction: $O \ominus 1 = \{E\}$ and $E \ominus 1 = \{1, O\}$.

Shape Analysis: Abstract Transfer Functions

- $\llbracket x := \text{null} \rrbracket^\sharp(G) = \text{norm}(G[\text{env}(x) \mapsto \text{null}])$.
- $\llbracket x := \text{new} \rrbracket^\sharp(G) = \text{Add fresh node } w, \text{succ}(w) = \text{null}, \text{len}(w) = 1, \text{gp}(w) = \text{ff}, \text{ set } x \text{ to } w$.
- $\llbracket x := y.n \rrbracket^\sharp(G) = \text{error if } \text{env}(y) = \text{null}.$ Sets x to $\text{succ}(\text{env}(y))$ if $\text{len} = 1$. If $\text{len} \in \{\text{O}, \text{E}\}$, materialises a fresh node of length $\text{len} \ominus 1$.
- $\llbracket x.n := y \rrbracket^\sharp(G) = \text{error if } \text{env}(x) = \text{null}.$ Detaches $\text{env}(x)$ and conditionally attaches to $\text{env}(y)$.
- Reports sharing if $\text{env}(y)$ has a predecessor or $\text{gp} = \text{tt}$, and reports a cycle if $\text{env}(y) \rightarrow^* \text{env}(x)$.

Implementation Details

- The analyses are implemented in Python 3 using only the standard library, in `impl/parity/parity.py` and `impl/shape/shape.py`.
- A custom tokenizer and parser handle the CFG syntax, accommodating spaces, comments, and varying assume block formatting.
- In the Parity Analysis, states are represented as frozensets of tuples over $\{Z, E, O\}$. The pointwise domain is achieved by expanding states, applying the exact transformer, and projecting them to secure the best transformers.
- In the Shape Analysis, graphs are canonicalised by setting the node name as the set of pointing variables. This guarantees that isomorphic graphs coincide cleanly.
- Both engines utilize chaotic iteration.

Demo Output: Parity & Shape Analysis

The following examples highlight the console output during analysis:

```
$ python3 impl/parity/parity.py impl/parity/examples/example-int.txt
```

```
L6: {(n:E, i:E, j:E), (n:0, i:0, j:0), (n:Z, i:Z, j:Z)}
```

```
RESULT: VERIFIED all 2 assertion(s) hold on every execution.
```

```
$ python3 impl/shape/shape.py impl/shape/examples/example-shape.txt
```

```
L42: 6 graphs:
```

```
{x,xx} -1-> {p,z} -1-> NULL; {y,yy} -1-> NULL NULL: t,zz
```

```
{x} -0-> {xx} -1-> {p,z} -1-> NULL; {y} -0-> {yy} -1-> NULL ...
```

```
RESULT: VERIFIED memory safe, no cycles, no sharing, and all 9 assertion(s) hold.
```

```
$ python3 impl/shape/shape.py impl/shape/examples/example-shape-variant1.txt
```

```
ERROR line 37 [L43 yy.n := z L50]: sharing: the cell pointed to by z is already pointed to by an n-field
```